参考答案（先做完再看）

每个场景一份完整脚本。对答案顺序：先看"先想什么"（思路对不对）→"为什么这么写"（取舍对不对）→最后看代码（细节对不对）。

场景 1｜批量文件重命名工具

① 先想什么：先收集普通文件，并在比较扩展名时统一转小写；再生成完整计划。正式执行和 `--dry-run` 都先做同一套完整校验：检查源文件、目标名重复和目标占用，任何冲突都整体拒绝；如果所有源名已经等于目标名，则报告“已完成”并以 0 退出。执行中异常时回滚已完成项目。

（回滚属于拓展：只做“冲突整体拒绝、不覆盖、重复运行幂等”也达到本场景的主线标准；会回滚则在验收中另记证据）

② 为什么这么写：`lower()` 使 `.jpg` 与 `.JPG` 一致；`os.path.isfile()` 排除目录；先验证完整计划再执行，避免改到一半才发现冲突。正式操作先把完整计划做静态预检，再用同目录硬链接 `os.link(..., follow_symlinks=False)` 原子创建目标，成功后 `os.unlink` 移除源名；即使检查后外部程序抢先创建同名目标，链接步骤也会失败而不会覆盖。该写法要求源、目标在同一文件系统（本例同目录）；回滚使用同一不覆盖函数。

③ 完整代码（rename_tool.py）：

```
import argparse
import os
import sys


def collect_files(directory, ext):
    ext = ext.lower().lstrip(".")
    result = []
    try:
        names = os.listdir(directory)
    except OSError as exc:
        print(f"读取目录失败：{exc}")
        return result
    for name in names:
        path = os.path.join(directory, name)
        if os.path.isfile(path) and name.lower().endswith("." + ext):
            result.append(name)
    return sorted(result)


def build_plan(files, prefix):
    return [
        (old, f"{prefix}_{number:03d}.{old.rsplit('.', 1)[-1]}")
        for number, old in enumerate(files, 1)
    ]


def validate_plan(directory, plan):
    # 列表推导式：把"循环 + 条件 + 收集"写成一行；本教材未单独教—用 for 循环等价写出、结果一样也算过
    targets = [new for old, new in plan if old != new]
    if len(targets) != len(set(targets)):
        return False, "生成的目标名重复"
    for old, new in plan:
        source = os.path.join(directory, old)
        target = os.path.join(directory, new)
        if not os.path.isfile(source):
            return False, f"源文件不存在或不是普通文件：{old}"
        if old != new and os.path.exists(target):
            return False, f"目标已存在：{new}"
    return True, ""


def move_no_replace(source, target):
```

```
    # 先原子创建目标链接；目标已被抢占时 os.link 直接失败，不会覆盖。
    os.link(source, target, follow_symlinks=False)
    os.unlink(source)


def rename_files(directory, plan):
    ok, reason = validate_plan(directory, plan)
    if not ok:
        print(f"拒绝执行：{reason}")
        return False
    completed = []
    try:
        for old, new in plan:
            if old == new:
                continue
            move_no_replace(os.path.join(directory, old), os.path.join(directory, new))
            completed.append((old, new))
            print(f"[完成] {old} -> {new}")
    except OSError as exc:
        print(f"执行中断：{exc}")
        for old, new in reversed(completed):
            old_path = os.path.join(directory, old)
            new_path = os.path.join(directory, new)
            if os.path.exists(new_path) and not os.path.exists(old_path):
                move_no_replace(new_path, old_path)
        print("已尝试恢复已完成项目，请重新检查目录")
        return False
    return True


def main():
    parser = argparse.ArgumentParser(description="批量文件重命名工具")
    parser.add_argument("directory", help="要整理的目录路径")
    parser.add_argument("--ext", default="jpg", help="扩展名，不区分大小写")
    parser.add_argument("--prefix", default="photo", help="新文件名前缀")
    parser.add_argument("--dry-run", action="store_true", help="只打印计划，不真正改名")
    args = parser.parse_args()
    if not os.path.isdir(args.directory):
        print(f"目录不存在或不是目录：{args.directory}")
        return 1
    files = collect_files(args.directory, args.ext)
    if not files:
        print("没有找到目标普通文件，或目录不可读")
        return 1
    plan = build_plan(files, args.prefix)
    ok, reason = validate_plan(args.directory, plan)
    if not ok:
        print(f"拒绝执行：{reason}")
        return 1
    for old, new in plan:
        print(f"[计划] {old} -> {new}")
    if all(old == new for old, new in plan):
        print("已完成：所有文件名都已符合目标规则")
        return 0
    if args.dry_run:
        print(f"共 {len(plan)} 个普通文件，全部通过冲突预检（未执行）")
        return 0
    if not rename_files(args.directory, plan):
        return 1
```

```
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

④ **怎么验证**：先记录测试目录的名称、数量和文件内容。加入 .jpg、.JPG、同名目录、空目录、权限异常等样例；先运行 --dry-run，它必须和正式执行使用同一套冲突预检。确认计划后再正式执行；目标冲突、源文件消失或权限异常时应整体拒绝或恢复，不能留下部分改名。最后再运行一次：若 9 个文件已经全部等于目标名，程序应打印“已完成”并以退出码 0 结束（幂等），而不是统一报“目标已存在”。

场景 2｜天气查询器

① **先想什么**：城市 → 经纬度 → URL → 请求 → 状态码 → JSON → 字段校验 → 输出；网络层、HTTP 层和数据格式层分别处理。
② **为什么这么写**：HTTP 200 不代表 JSON 或字段结构正确，因此处理网络失败、非 200、JSON 解析失败、缺字段和字段类型错误。
③ **完整代码**（weather.py）：

```
import argparse
import requests
import sys

CITIES = {
    "北京": (39.90, 116.40),
    "上海": (31.23, 121.47),
    "杭州": (30.25, 120.17),
    "广州": (23.13, 113.26),
    "成都": (30.57, 104.06),
}


def weather_text(code):
    if code == 0:
        return "晴"
    if code in (1, 2):
        return "多云"
    if code == 3:
        return "阴"
    if code == 45:
        return "雾"
    if 51 <= code <= 67 or 80 <= code <= 82:
        return "有雨"
    if 71 <= code <= 77:
        return "有雪"
    if code >= 95:
        return "雷暴"
    return "未知"


def main():
    parser = argparse.ArgumentParser(description="命令行天气查询器")
    parser.add_argument("city", help="城市名")
    args = parser.parse_args()

    if args.city not in CITIES:
        print(f"暂不支持城市：{args.city}，可选：{'、'.join(CITIES)}")
        return

    latitude, longitude = CITIES[args.city]
    url = (
```

```
        "https://api.open-meteo.com/v1/forecast?"
        f"latitude={latitude}&longitude={longitude}&current_weather=true"
    )

    try:
        response = requests.get(url, timeout=10)
        if response.status_code != 200:
            print(f"接口返回错误：状态码 {response.status_code}")
            return
        try:
            data = response.json()
        except ValueError as exc:
            print(f"接口数据格式错误：不是有效 JSON（{exc}）")
            return
    except requests.exceptions.RequestException:
        print("网络不通，请检查网络后重试")
        return

    current = data.get("current_weather") if isinstance(data, dict) else None
    if not isinstance(current, dict):
        print("接口数据格式错误：缺少 current_weather 对象")
        return

    required = ("temperature", "weathercode", "windspeed")
    # 列表推导式：等价于"for key in required: 满足条件就收进列表"；没学过可直接写 for 循环
    missing = [key for key in required if key not in current]
    if missing:
        print(f"接口数据格式错误：缺少字段 {', '.join(missing)}")
        return

    temperature = current["temperature"]
    code = current["weathercode"]
    windspeed = current["windspeed"]
    valid_number = (int, float)
    if (
        not isinstance(temperature, valid_number)
        or isinstance(temperature, bool)
        or not isinstance(code, int)
        or isinstance(code, bool)
        or not isinstance(windspeed, valid_number)
        or isinstance(windspeed, bool)
    ):
        print("接口数据格式错误：天气字段类型不符合预期")
        return

    line = f"{args.city}当前温度 {temperature}°C，{weather_text(code)}，风速 {windspeed} km/h"
    print(line)
    with open("weather_report.txt", "a", encoding="utf-8") as fh:
        fh.write(line + "\n")
    print("已写入 weather_report.txt")


if __name__ == "__main__":
    sys.exit(main())
```

④ 怎么验证：正常接口、未知城市、断网/超时、非 JSON 的 200、缺字段和类型错误都应无 Traceback，并分别给出明确提示；连续跑两次后 `cat weather_report.txt` 应看到两行记录（判定里的"结果能存文件"就是它）。

场景 3｜本机巡检工具雏形

① 先想什么：三个检查函数 + 报告 + 日志。三个检查函数的完整代码见下方③；它们都按 subprocess 调用模板分别解析输出，但 `free -m` 输出单位是 MB（纯数字），第二行（索引 1）是 Mem 内存行；check_cpu 只取 `uptime` 末尾的 load average（负载平均值）。报告 = 三行结果拼起来写入文件；日志 = INFO 记开始/完成。
② 为什么这么写：内存百分比用 `used * 100 // total`（整数除法，教科书第 1 章）；负载数字在输出末尾 `load average:` 之后，用 rsplit 取最后一段再 strip；文件名日期用 time.strftime；日志用 logging.basicConfig。本关 v0 只处理 Ubuntu 正常命令输出；命令失败兜底和命令行阈值放到 7-02，README 放到 7-06，避免同一关混入尚未教学的验收项。
③ 完整代码（server_check.py；先 cd ~/lab 再运行，报告和日志都写进练习目录）：

```
import logging
import os
import subprocess
import time
import sys

logging.basicConfig(filename="server_check.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

def check_disk():  # 磁盘：df -h 根分区使用率
    r = subprocess.run(["df", "-h"], capture_output=True, text=True)
    for line in r.stdout.splitlines():
        parts = line.split()
        if len(parts) >= 6 and parts[5] == "/":
            percent = int(parts[4].strip("%"))
            status = "告警" if percent > 80 else "正常"
            return f"磁盘：使用率 {percent}%，状态：{status}"
    return "磁盘：未找到根分区信息"

def check_memory():  # 内存：free -m 第二行 Mem
    r = subprocess.run(["free", "-m"], capture_output=True, text=True)
    parts = r.stdout.splitlines()[1].split()
    total = int(parts[1])
    used = int(parts[2])
    percent = used * 100 // total
    status = "告警" if percent > 80 else "正常"
    return f"内存：使用率 {percent}%，状态：{status}"

def check_cpu():  # 负载：1 分钟负载超过 CPU 核数就告警
    r = subprocess.run(["uptime"], capture_output=True, text=True)
    load = r.stdout.strip().rsplit("load average:", 1)[-1].strip()
    one_min = float(load.split(",")[0])
    cores = os.cpu_count() or 1
    threshold = float(cores)
    status = "告警" if one_min > threshold else "正常"
    return f"负载（1/5/15分钟）：{load}；1分钟阈值 {threshold:.1f}，状态：{status}"

def main():
    logging.info("开始巡检")
    report = "\n".join([check_disk(), check_memory(), check_cpu()])
    print(report)
    filename = "server_report_" + time.strftime("%Y-%m-%d") + ".txt"
    with open(filename, "w", encoding="utf-8") as f:
        f.write(report + "\n")
    print("报告已写入", filename)
    logging.info("巡检完成，报告写入 %s", filename)

if __name__ == "__main__":
    sys.exit(main())
```

④ **怎么验证**：先 cd ~/lab，记录 server_check.log 当前行数或尾部时间戳；运行后应打印三行检查结果并生成 server_report_今天日期.txt，报告内也是同样三行。只核对**本次新增**的一条“开始巡检”和一条“巡检完成”，不要固定断言日志总行数；重复运行前重新记录起点。自测阈值：把 percent > 80 临时改成 percent > 0，应输出“告警”，验证后立即改回。最后删除临时报告；若不保留练习日志，可在确认本次记录后删除 server_check.log。Windows 跑 df/free/uptime 会报 **FileNotFoundError**——正常，本题标了 。

场景 4 | Nginx 日志分析器

① **先想什么**：逐行读取 → 分层校验 IP、引号片段、请求字段和状态码 → 正常行计数 → 坏行记录行号和原因并继续 → 输出报告。
② **为什么这么写**：日志可能有空行、缺引号、短请求行或非数字状态码。解析函数不能只保护引号片段，还要保护请求字段和状态码；坏行跳过并计数，避免一条坏日志拖垮整批分析。
③ **完整代码**（log_analyzer.py）：

```
import argparse
import sys


def parse_line(line):
    fields = line.split()
    if not fields:
        return None, "空行"
    ip = fields[0]
    segments = line.split('"')
    if len(segments) < 3:
        return None, "缺少完整引号片段"
    request_parts = segments[1].split()
    if len(request_parts) < 2:
        return None, "请求行字段不完整"
    status_parts = segments[2].split()
    if not status_parts or not status_parts[0].isdigit():
        return None, "状态码缺失或不是数字"
    return (ip, request_parts[1], status_parts[0]), ""


def add_count(data, key):
    data[key] = data.get(key, 0) + 1


def top_items(data, number):
    # 零基础写法（不用 key= 和 lambda——那俩本教材未教）：
    # 把 (IP, 次数) 翻成 (次数, IP) 元组，再 sorted(reverse=True)——
    # 元组比较先看第一个元素，所以次数大的排前面；同次数再按 IP 排。
    flipped = []
    for ip, count in data.items():
        flipped.append((count, ip))
    flipped = sorted(flipped, reverse=True)
    # reverse=True = 降序（从大到小）；不写就是从"最小"开始
    result = []
    for count, ip in flipped[:number]:
        result.append((ip, count))
    return result


def main():
    parser = argparse.ArgumentParser(description="Nginx 日志分析器")
    parser.add_argument("logfile", help="日志文件路径")
    args = parser.parse_args()
    ip_count, url404, bad_count = {}, {}, 0
    try:
```

```
        source = open(args.logfile, "r", encoding="utf-8")
    except FileNotFoundError:
        print(f"日志文件不存在：{args.logfile}")
        return 1
    except OSError as exc:
        print(f"无法读取日志文件：{exc}")
        return 1
    with source:
        for line_number, raw_line in enumerate(source, 1):
            parsed, reason = parse_line(raw_line.strip())
            if parsed is None:
                bad_count += 1
                print(f"[跳过] 第 {line_number} 行：{reason}")
                continue
            ip, url, status = parsed
            add_count(ip_count, ip)
            if status == "404":
                add_count(url404, url)
    report = ["== 访问量 Top 10 IP =="]
    report.extend(f"{ip}  {count} 次" for ip, count in top_items(ip_count, 10))
    report.append("")
    report.append("== 404 最多的 5 个 URL ==")
    report.extend(f"{url}  {count} 次" for url, count in top_items(url404, 5))
    report.append("")
    report.append(f"坏行：{bad_count}")
    text = "\n".join(report)
    print(text)
    try:
        with open("report.txt", "w", encoding="utf-8") as file:
            file.write(text + "\n")
    except OSError as exc:
        print(f"报告写入失败：{exc}")
        return 1
    print("报告已写入 report.txt")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

④ 怎么验证：在样例中加入空行、短请求行、缺引号、缺状态码和非数字状态码；程序不应崩溃，正常行统计仍应正确，并逐行提示被跳过的原因，报告中的“坏行”数量应与实际一致。正常样例仍应得到 66.249.66.1 为 Top 1（5 次）及 4 个 404 URL。完成后删除 `report.txt` 和临时日志。

场景 5 ☆ | paramiko 远程巡检雏形（选做）

① 先想什么：连接、认证、执行命令和释放连接是四个边界。任何一步失败都要给出可定位信息；每条远程命令都同时读取 stdout、stderr，并检查返回码。
② 为什么这么写：`getpass.getpass` 输入密码时不回显；`timeout` 防止连接无限等待；`try/finally` 保证连接关闭；`recv_exit_status()` 区分“有输出但失败”和真正成功。`AutoAddPolicy` 仅限自己的练习虚拟机，生产环境必须核验主机密钥。
③ 完整代码（remote_check.py）：
安装依赖：进入本册前置创建并激活的 `.venv` 后执行 `pip install paramiko`；未安装时会报 **`ModuleNotFoundError: No module named 'paramiko'`**。只连自己的练习虚拟机。

```
import getpass
import paramiko
import sys


def main():
```

```
    host = input("Ubuntu IP：").strip()
    user = input("用户名：").strip()
    password = getpass.getpass("密码（不回显）：")
    while True:
        try:
            port = int(input("SSH 端口（默认 22）：").strip() or "22")
            if not 1 <= port <= 65535:
                raise ValueError
            break
        except ValueError:
            print("端口必须是 1~65535 的整数，请重新输入")
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    try:
        client.connect(host, port=port, username=user, password=password,
                       timeout=10, auth_timeout=10, banner_timeout=10)
        for command in ["df -h", "free -m", "uptime"]:
            print("=" * 10, command, "=" * 10)
            stdin, stdout, stderr = client.exec_command(command)
            output = stdout.read().decode("utf-8", errors="replace")
            error = stderr.read().decode("utf-8", errors="replace")
            # errors="replace"：解码遇到坏字节就替换成符号、不抛异常—远程输出有乱码时脚本照样跑
            code = stdout.channel.recv_exit_status()
            print("stdout：", output or "（空）")
            print("stderr：", error or "（空）")
            print("退出码：", code)
            if code != 0:
                print("远程命令失败，请根据 stderr 排查")
    except paramiko.AuthenticationException:
        print("认证失败：用户名或密码错误")
    except (paramiko.SSHException, OSError) as exc:
        print(f"SSH 连接或协议失败：{exc}")
    finally:
        client.close()
        print("SSH 连接已关闭")


if __name__ == "__main__":
    sys.exit(main())
```

④ **怎么验证**：在自有虚拟机测试 SSH 未启动、认证失败和远程命令失败；再输入一个明确未监听的端口（例如 9）验证错误端口分支，脚本应显示连接拒绝或超时并友好结束。隐藏密码，并由 `finally` 关闭连接。成功命令核对 stdout、stderr、退出码。没有 SSH 条件可跳过，不影响主线。

附录 C | 场景 → 产出 → 保存位置

场景	脚本文件	保存到	对应产出
场景 1	rename_tool.py	python-scripts	脚本 1
场景 2	weather.py	python-scripts	脚本 2
场景 3	server_check.py	server-health-check（新建）	王牌项目版本 0（只含三项检查、报告、日志）
场景 4	log_analyzer.py	python-scripts	脚本 3（主线用样例日志；Docker 真实版为外部扩展）
场景 5	remote_check.py	server-health-check	版本 2 雏形（选做）

卷末速查 · 五个场景各要什么

场景	模块	关键写法
1 批量重命名	argparse · os · os.path	listdir / link / unlink / join / exists（原子创建目标，再移除源名）
2 天气查询	requests	get(url, timeout=10) → status_code → .json()
3 本机巡检	subprocess · logging · time	run([...], capture_output=True, text=True) / basicConfig / strftime
4 日志分析	内置为主	open（**逐行遍历**） / split / strip / get / items / sorted
5 SSH 巡检（选做）	paramiko · getpass	SSHClient / AutoAddPolicy / connect / exec_command

阶段总结练习（打完本阶段关卡再做）

阶段 3 单一验收卡：能力目标是“把三个只读命令解析成可留档的巡检报告”；必交证据是 `server_check.py`、一份报告样例、一次开始/完成日志；本关不要求 argparse、命令失败兜底或 README，这三项分别在 7-02、7-06 验收。

阶段 3 • 学习总结练习：本机巡检 v0（subprocess 查磁盘 / 内存 / 负载）

业务场景：领导问"这台机器现在状态怎么样？"——你要交的不是截图，而是一份能自动生成的巡检报告。

综合要求（只收 v0）：① 用 subprocess 调 df -h / free -m / uptime 取数；② 三个检查函数各自按 v0 固定口径给出正常/告警状态（磁盘与内存 >80%，1 分钟负载 >CPU 核数）；③ 结果写成带日期的报告，并用 logging 记录开始/完成；④ 在同一实验目录用 `python3 server_check.py` 一次跑出报告。

验收：· 一条命令能跑出三行结果和报告文件 · 当前环境触发阈值时能显示告警 · 日志里本次有开始/完成记录 · 能讲清三个检查函数各在查什么

产出与后续：`server-health-check/server_check.py` v0 + 一份报告样例。命令行阈值、命令失败兜底在 7-02；README 与设计决策在 7-06；这两项不属于 3-17。

AI 运维指向：这就是"服务器健康巡检"的骨架；再加 GPU 指标（nvidia-smi）和远程执行，就是 AI 集群巡检。

本关证据：脚本、报告样例和日志各一份就够；证据都来自 v0 已教过的能力。